

Italian Restaurant Sales Analytics

05 - Products Data Cleaning and Preparation

Objective

This notebook cleans and validates the **Products** worksheet from the Italian Restaurant Sales Analytics dataset. It verifies data quality, checks for missing values and duplicates, validates data types and product categories, and exports a cleaned dataset for further analysis in Python, SQL, and Power BI.

Step 1 - Import Libraries

The first step is to import the Python libraries required for data manipulation and analysis.

```
In [1]: # Import pandas for data manipulation and analysis  
import pandas as pd
```

Step 2 - Load the Excel Workbook

Define the location of the Excel workbook.

```
In [2]: # Define the relative path to the Excel workbook  
file_path = "../data/raw/restaurant-sales-analytics-data.xlsx"
```

Step 3 - Read the Products Worksheet

Load the Products worksheet into a pandas DataFrame.

```
In [3]: # Load the Products worksheet into a pandas DataFrame  
products = pd.read_excel(file_path, sheet_name="Products")
```

Step 4 - Preview the Dataset

Display the first five rows of the Products dataset.

```
In [4]: # Display the first five rows  
products.head()
```

	Category	Product	Quantity	Quantity (%)	Sales	Sales (%)
0	Pasta	Chicken Parmegiana	426.5	0.528218	6467.60	2.045481
1	Pasta	Lasagna	470.0	0.582093	6465.16	2.044709
2	Pasta	Carbonara	492.0	0.609340	6334.68	2.003443
3	Pasta	Ravioli Di Ricotta/Esp	424.3	0.525493	5096.16	1.611741
4	Pasta	Grilled Steak	355.0	0.439666	4819.62	1.524281

Step 5 - Inspect the Dataset Structure

Display the structure of the dataset, including data types, number of non-null values, and memory usage.

```
In [5]: # Display dataset information
products.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 343 entries, 0 to 342
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Category        343 non-null   object
1   Product         343 non-null   object
2   Quantity        343 non-null   float64
3   Quantity (%)    343 non-null   float64
4   Sales           343 non-null   float64
5   Sales (%)       343 non-null   float64
dtypes: float64(4), object(2)
memory usage: 16.2+ KB
```

Step 6 - Check Dataset Dimensions

Display the number of rows and columns in the dataset.

```
In [6]: # Display dataset dimensions
products.shape
```

```
Out[6]: (343, 6)
```

Step 7 - Check Missing Values

Identify whether the dataset contains any missing values in each column.

```
In [7]: # Check for missing values
products.isnull().sum()
```

```
Out[7]: Category      0
        Product      0
        Quantity     0
        Quantity (%)  0
        Sales        0
        Sales (%)    0
        dtype: int64
```

Step 8 - Check for Duplicate Rows

Identify whether the dataset contains duplicate records that may affect the analysis.

```
In [8]: # Check for duplicate rows
        products.duplicated().sum()
```

```
Out[8]: np.int64(0)
```

Step 9 - Display Column Names

Display all column names in the dataset.

```
In [9]: # Display column names
        products.columns
```

```
Out[9]: Index(['Category', 'Product', 'Quantity', 'Quantity (%)', 'Sales',
              'Sales (%)'],
              dtype='object')
```

Step 10 - Display Summary Statistics

Display descriptive statistics for the numerical columns to better understand the distribution of the data.

```
In [10]: # Display summary statistics
         products.describe()
```

Out[10]:

	Quantity	Quantity (%)	Sales	Sales (%)
count	343.000000	343.000000	343.000000	343.000000
mean	124.503110	0.154196	896.113907	0.283410
std	227.716185	0.282025	1578.665789	0.499278
min	1.000000	0.001238	0.000000	0.000000
25%	4.000000	0.004954	11.400000	0.003605
50%	44.000000	0.054494	111.320000	0.035207
75%	146.500000	0.181440	1105.200000	0.349537
max	1930.000000	2.390295	9538.150000	3.016591

Step 11 - Check Unique Categories

Display all unique product categories to verify that they are correctly classified.

```
In [11]: # Display unique product categories
products["Category"].unique()
```

```
Out[11]: array(['Pasta', 'Meat, chicken & seafood', '3x2 Promotion', 'Entrees',
        'Specials', 'Juice', 'Pizza & Calzone', 'Risotto', 'Salads',
        'Water', 'Extras', 'Soups', 'Italian Lunch', 'Dessert',
        'Pop Beverages', 'Coffee & Tea', 'Events Menus', 'Children',
        'Beer', 'Wine', 'Liquor'], dtype=object)
```

Step 12 - Count Unique Categories

Count the number of unique product categories in the dataset.

```
In [12]: # Count unique product categories
products["Category"].nunique()
```

Out[12]: 21

Step 13 - Count Unique Products

Count the number of unique products in the dataset.

```
In [13]: # Count unique products
products["Product"].nunique()
```

Out[13]: 342

Step 14 - Display Data Types

Display the data types of all columns.

```
In [14]: # Display data types
products.dtypes
```

```
Out[14]: Category      object
Product      object
Quantity      float64
Quantity (%)  float64
Sales         float64
Sales (%)     float64
dtype: object
```

Step 15 - Check for Negative Sales

Verify whether the dataset contains any negative sales values.

```
In [15]: # Check for negative sales
(products["Sales"] < 0).sum()
```

```
Out[15]: np.int64(0)
```

Step 16 - Check for Zero Sales

Verify whether the dataset contains any products with zero sales.

```
In [16]: # Check for zero sales
(products["Sales"] == 0).sum()
```

```
Out[16]: np.int64(55)
```

Step 17 - Check for Negative Quantity

Verify whether the dataset contains any negative quantities.

```
In [17]: # Check for negative quantities
(products["Quantity"] < 0).sum()
```

```
Out[17]: np.int64(0)
```

Step 18 - Check for Zero Quantity

Verify whether the dataset contains any products with zero quantity sold.

```
In [18]: # Check for zero quantities
(products["Quantity"] == 0).sum()
```

```
Out[18]: np.int64(0)
```

Step 19 - Create a Clean Copy

Create a copy of the cleaned dataset to preserve the original data before exporting.

```
In [19]: # Create a clean copy of the dataset
products_cleaned = products.copy()
```

Step 20 - Verify the Clean Dataset

Display the first rows of the cleaned dataset to verify that it is ready for export.

```
In [20]: # Preview the cleaned dataset
products_cleaned.head()
```

```
Out[20]:
```

	Category	Product	Quantity	Quantity (%)	Sales	Sales (%)
0	Pasta	Chicken Parmegiana	426.5	0.528218	6467.60	2.045481
1	Pasta	Lasagna	470.0	0.582093	6465.16	2.044709
2	Pasta	Carbonara	492.0	0.609340	6334.68	2.003443
3	Pasta	Ravioli Di Ricotta/Esp	424.3	0.525493	5096.16	1.611741
4	Pasta	Grilled Steak	355.0	0.439666	4819.62	1.524281

Step 21 - Export the Clean Dataset

Export the cleaned Products dataset to the cleaned data folder.

```
In [21]: # Export the cleaned dataset
products_cleaned.to_excel(
    "../data/cleaned/products_cleaned.xlsx",
    index=False
)

print("Products dataset exported successfully!")
```

Products dataset exported successfully!

Step 22 - Cleaning Summary

Display a summary of the data cleaning process.

```
In [22]: print("===== CLEANING SUMMARY =====")

print(f"Rows: {products.shape[0]}")
```

```

print(f"Columns: {products.shape[1]}")

print(f"Missing values: {products.isnull().sum().sum()}")
print(f"Duplicate rows: {products.duplicated().sum()}")

print(f"Unique Categories: {products['Category'].nunique()}")
print(f"Unique Products: {products['Product'].nunique()}")

print(f"Negative Sales: {(products['Sales'] < 0).sum()}")
print(f"Zero Sales: {(products['Sales'] == 0).sum()}")

print(f"Negative Quantity: {(products['Quantity'] < 0).sum()}")
print(f"Zero Quantity: {(products['Quantity'] == 0).sum()}")

print("Clean dataset exported successfully.")
print("=====")

```

```

===== CLEANING SUMMARY =====
Rows: 343
Columns: 6
Missing values: 0
Duplicate rows: 0
Unique Categories: 21
Unique Products: 342
Negative Sales: 0
Zero Sales: 55
Negative Quantity: 0
Zero Quantity: 0
Clean dataset exported successfully.
=====

```

Author

Guido Acosta

Data Analyst Portfolio Project

GitHub Portfolio – 2026